Knowledge Checks – Part 2

This page covers the material from Midterm 1 through Midterm 2.

The intent of this page is to give you practice questions like you will see on the midterm. You should attempt these with no notes, references or AI assistance, as you won't have those on the midterm.

> Knowledge checks are updated to cover material from Midterm 1 to Midterm 2.

- Ownership and Borrowing
- Strings and Vecs
- Slices
- Structs
- Methods
- Generics
- Option and Result
- Closures
- Iterators
- Iterators and Closures Combined
- Modules and Crates
- Tests

Reference Material (Also Provided in the Midterm)

> Do you want to make a case for adding more reference material? Post on Piazza and I'll consider it.

Iterator Methods and Adapters

- `next(&mut self) -> Option<Self::Item>` Get the next element of an iterator (`None` if there isn't one)
- `collect<B>(self) -> B` -> Transforms the iterator into a collection of type `B`
- `take(N)` -> Take first N elements of an iterator and *turn them into an iterator*
- `cycle()` -> Turn a finite iterator into an infinite one that repeats itself
- `for_each(|x| ...)` -> Apply a closure to each element in the iterator
- `filter(|x| ...)` -> *Create new iterator* from old one for elements where closure is true
- `map(|x| ...)` -> *Create new iterator* by applying closure to input iterator
- `any(|x| ...)` -> Return `true` if closure is true for any element of the iterator
- `fold(init, |acc, x| ...)` -> Initialize accumulator to `init`, execute closure on each element
- `reduce(|acc, x| ...)` -> Similar to fold but the initial value is the first element (returns `Option`)
- `zip(other_iter)` -> Zip two iterators together into pairs of `(a, b)`
- `enumerate()` -> Create iterator of `(index, element)` pairs
- `partition(|x| ...)` -> Split iterator into two *collections* based on predicate
- `sum()` -> Sum all elements (works on numeric iterators)
- `count()` -> Count the number of elements in the iterator
- `copied()` -> Create iterator of owned copies from an iterator of references

String and &str Methods

- `s.len()` -> Number of bytes in the string
- `s.contains("sub")` -> `true` if string contains the substring
- `s.to_uppercase()` -> Returns a new `String` in uppercase
- `s.to_lowercase()` -> Returns a new `String` in lowercase
- `s.push('c')` -> Append a single character (mutates `String`)
- `s.push_str("text")` -> Append a string slice (mutates `String`)
- `s.chars()` -> Iterator over the characters of the string
- `s.split_whitespace()` -> Iterator of substrings split by whitespace
- `s.parse::<T>()` -> Parse the string into type `T`, returns `Result<T, E>`
- `s.is_empty()` -> `true` if the string has length 0

Vec Methods

- `v.push(val)` -> Append element to the end
- `v.pop()` -> Remove and return last element as `Option<T>`
- `v.get(i)` -> Return `Option<&T>` for safe bounds-checked access
- `v.len()` -> Number of elements
- `v.is_empty()` -> `true` if the vector has no elements
- `v.iter()` -> Iterator of immutable references `&T`
- `v.iter_mut()` -> Iterator of mutable references `&mut T`
- `v.into_iter()` -> Consuming iterator of owned values `T`
- `v.sort_by(|a, b| ...)` -> Sort in place using a comparison closure (closure returns `Ordering`)

Option Methods

- `opt.unwrap()` -> Extract the value or **panic** if `None`
- `opt.unwrap_or(default)` -> Extract the value or return `default` (eager)
- `opt.unwrap_or_else(|| ...)` -> Extract the value or compute default via closure (lazy)
- `opt.map(|x| ...)` -> Transform the inner value; `None` stays `None`
- `opt.is_some()` -> `true` if `Some`
- `opt.is_none()` -> `true` if `None`

Other Useful Methods

- `val.clone()` -> Create a deep copy of the value
- `a.cmp(&b)` -> Compare two values, returns `std::cmp::Ordering`
- `(x as f64).sqrt()` -> Square root of a floating-point number
- `format!("...", args)` -> Like `println!` but returns a `String` instead of printing
- `{:?}` -> Debug format specifier (in `println!` / `format!`)
- `{:.2}` -> Display with 2 decimal places

Test Assertion Macros

- `assert!(expr)` -> Pass if `expr` is `true`
- `assert_eq!(a, b)` -> Pass if `a == b`; shows both values on failure
- `assert_ne!(a, b)` -> Pass if `a != b`; shows both values on failure

- Assert macros can take an optional custom failure message, e.g., `assert_eq!(a, b, "{} is not equal to {}", a, b)`.

Ownership and Borrowing

Prerequisite: Complete Part 1

What are the three ownership rules in Rust?

This program has a bug and does not compile. Explain why and how to fix it.

```
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;
    println!("{}", s1);
}
```

Select all types that implement the `Copy` trait (i.e., are copied rather than moved on assignment):

- ☐ A. `i32`
- ☐ B. `String`
- ☐ C. `bool`
- ☐ D. `f64`
- ☐ E. `Vec<i32>`

Fix this program so it compiles and prints both strings.

```
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;
    println!("{} {}", s1, s2);
}
```

What is the difference between an immutable reference (`&T`) and a mutable reference (`&mut T`) in Rust? How many of each can you have at the same time?

This program has a bug and does not compile. Explain why and how to fix it.

```
fn main() {
    let mut s = String::from("hello");
    let r1 = &s;
    s.push_str(" world");
    println!("{}", r1);
```

```
    println!("{}", s);
}
```

Write a function `first_word_len` that takes an immutable reference to a string slice and returns the length of the first word (characters before the first space). If there is no space, return the length of the entire string.

Call the function from `main` with `"hello world"` and print the result.

This program has a bug and does not compile. Explain why and how to fix it.

```
fn take_ownership(s: String) {
    println!("{}", s);
}

fn main() {
    let s = String::from("hello");
    take_ownership(s);
    println!("{}", s);
}
```

Select all statements that are true about Rust's borrowing rules:

- ☐ A. You can have multiple immutable references at the same time.
- ☐ B. You can have multiple mutable references at the same time.
- ☐ C. You can have one mutable reference and one immutable reference at the same time.
- ☐ D. A mutable reference requires the variable itself to be declared `mut`.

What is the output of this program?

```
fn main() {
    let s1 = String::from("hello");
    let s2 = String::from("ds210!");
    {
        let s2 = String::from("world");
        println!("{} {}", s1, s2);
    }
    println!("{} {}", s1, s2);
}
```

Predict the output of this program.

```rust
fn append_suffix(mut s: String, suffix: &str) -> String {
    s.push_str(suffix);
    s
}

fn main() {
    let s1 = String::from("hello");
    let s2 = append_suffix(s1, ", world");
    println!("{}", s2);
}
```

Predict the output of this program.

```rust
fn main() {
    let mut v = vec![1, 2, 3];
    let r1 = &v;
    let r2 = &v;
    println!("{:?} {:?}", r1, r2);
    v.push(4);
    println!("{:?}", v);
}
```

When a `String` is assigned to another variable, ownership is _____ . To create a deep copy of a `String`, you call the `___` method.

Fix this program so it compiles. Do NOT use `.clone()`.

```rust
fn print_length(s: String) {
    println!("Length: {}", s.len());
}

fn main() {
    let s = String::from("hello");
    print_length(s);
    println!("{}", s);
}
```

Strings and Vecs

Prerequisite: Ownership and Borrowing

Create a `Vec<i32>` containing [1, 2, 3, 4, 5] using the `vec!` macro. Then push the value 6 onto it. Print the vector using debug formatting.

What is the difference between `v[2]` and `v.get(2)` when accessing elements of a `Vec`?

This program has a bug and does not compile. Explain why and how to fix it.

```
fn main() {
    let mut v = vec![1, 2, 3];
    let first = &v[0];
    v.push(4);
    println!("{}", first);
}
```

Write a function `sum_vec` that takes an immutable reference to a `Vec<i32>` and returns the sum of all elements as `i32`.

Call it from `main` with `vec![10, 20, 30]` and print the result.

What is the difference between `String` and `&str` in Rust?

Select all statements that are true about stack and heap memory:

- ☐ A. Values with a known, fixed size at compile time are stored on the stack.
- ☐ B. `String` data is stored entirely on the stack.
- ☐ C. `Vec<T>` stores its elements on the heap.
- ☐ D. Stack allocation is generally faster than heap allocation.
- ☐ E. An `i32` variable is stored on the heap.

What is the output of this program?

```
fn main() {
    let mut v = vec![10, 20, 30];
    let last = v.pop();
    println!("{:?}", last);
    println!("{:?}", v);
}
```

What is the output of this program?

```
fn main() {
    let v = vec![10, 20, 30];
    println!("{:?}", v.get(1));
```

```
    println!("{:?}", v.get(5));
}
```

Write a program that creates a `Vec<i32>` with values `[1, 2, 3, 4, 5]`, doubles every element in place, and prints the result.

Create a `String` with value `"Hello, World!"`. Use string methods to: 1. Print its length 2. Check if it contains `"World"` 3. Convert it to uppercase and print the result

---

What is the difference between `Vec::new()` and `Vec::with_capacity(10)`? Why would you prefer one over the other?

Slices

Prerequisite: Ownership and Borrowing

What is a slice in Rust, and how does it differ from owning the data?

Initialize an array to be `[10, 20, 30, 40, 50]`. Create a slice containing elements `[20, 30, 40]` and print it with debug formatting.

Given `let arr = [1, 2, 3, 4, 5];`:

- `&arr[0..3]` gives ___.
- `&arr[2..]` gives ___.
- `&arr[..2]` gives ___.
- `&arr[..]` gives ___.

This program has a bug and does not compile. Explain why and how to fix it and what the output would be once fixed.

```
fn main() {
    let mut arr = [1, 2, 3, 4, 5];
    let slice = &arr[1..3];
    slice[0] = 99;
    println!("{:?}", slice);
}
```

Write a function `double_slice` that takes a mutable slice `&mut [i32]` and doubles every element in it.

Call it from `main` with `[1, 2, 3, 4]` and print the array after the call.

Select all statements that are true about string slices (`&str`):

- ☐ A. `&str` can reference a string literal stored in the program's read-only memory.
- ☐ B. `&str` can reference a portion of a `String` on the heap.
- ☐ C. `&str` owns the string data it points to.
- ☐ D. A string literal `"hello"` has type `&str`.

Write a function `sum_slice(data: &[i32]) -> i32` that returns the sum of all elements in a slice. Test it from `main` by passing a slice of indices 1, 2 and 3 of an array `[10, 20, 30, 40, 50]`.

Given `let s = String::from("Hello, World!");`, create a string slice of just `"World"` and print it. Reminder: for this string, bytes line up with characters.

Given `let s = "Hello,  World!";`, create a new String `substring` that contains only the UTF-8 characters from positions 7 to 15. Print it.

Why is it generally preferred to accept `&[T]` (a slice) rather than `&Vec<T>` as a function parameter?

A slice is internally represented as two values: a `___` to the first element and a `___`. Unlike a `Vec`, a slice does not have a capacity field.

Structs

Prerequisite: Ownership and Borrowing

Define a struct `Student` with fields `name` (String), `age` (u32), and `gpa` (f64). In `main`, create a `Student` and print each field.

Define a tuple struct `Color` with three `u8` fields (for red, green, blue). Create an instance initialized to values 128, 111, 154, and print each component.

What is the output of this program?

```rust
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 10, y: 20 };
    let Point { x, y } = p;
    println!("x={}, y={}", x, y);
}
```

Select all statements that are true about structs and tuple structs:

- ☐ A. Named struct fields are accessed with dot notation and the field name.
- ☐ B. Tuple struct fields are accessed with dot notation and a numeric index.
- ☐ C. You can omit fields when creating a named struct instance.
- ☐ D. Tuple structs provide type safety over plain tuples with the same field types.

Define an enum `Shape` with variants: - `Circle` with an `f64` field `radius` - `Rectangle` with an `f64` field `width` and an `f64` field `height`

Write a function `area(shape: &Shape) -> f64` that returns the area.

Write a main function that creates a circle with radius 5.0 and rectangle with width 4.0 and height 3.0, and prints the area of each.

Hint: You can access the value of π using `std::f64::consts::PI`.

This program has a bug and does not compile. Explain why and how to fix it.

Hint: Think about ownership rules.

```rust
#[derive(Clone)]
struct Book {
    title: String,
    pages: u32,
```

```
}

fn main() {
    let b1 = Book { title: String::from("Rust"), pages: 300 };
    let b2 = b1;
    println!("{}", b1.title);
}
```

Define a struct `Temperature` with a field `celsius: f64`. Write a function `to_fahrenheit(temp: &Temperature) -> f64` that converts Celsius to Fahrenheit using the formula `F = C * 9/5 + 32`. Test it in `main`.

Methods

Prerequisite: Structs

Implement the following code: 1. Define a struct `Rectangle` with `width` and `height` (both `f64`). 2. Implement a constructor `new` that takes `width` and `height` and returns a new `Rectangle`. 3. Implement a method `area` that returns the area. 4. Write a `main` that creates a rectangle, using the constructor method, of width 10.0 and height 5.0 and prints its area.

Explain the difference between `&self`, `&mut self`, and `self` as method parameters.

Define a struct `Counter` with a field `count: i32`. Implement three methods: 1. `new` – associated function returning a Counter with count 0 2. `increment` – adds 1 to count 3. `value` – returns the current count (as an `i32`)

Test it in `main` by creating a counter, incrementing it three times, and printing the result.

This program has a bug and does not compile. Explain why and how to fix it.

```
struct Ticket {
    event: String,
}

impl Ticket {
    fn redeem(self) {
        println!("Redeemed: {}", self.event);
```

```
    }
}

fn main() {
    let t = Ticket { event: String::from("Concert") };
    t.redeem();
    t.redeem();
}
```

Select all statements that are true about methods and associated functions:

- ☐ A. Methods take `self`, `&self`, or `&mut self` as their first parameter.
- ☐ B. You can have associated functions, like constructors, that do not have a `self` parameter.
- ☐ C. Methods are called with dot syntax: `instance.method()`.
- ☐ D. Associated functions of a struct that has not been instantiated are called with dot syntax: `instance.function()`.

Can you have more than one `impl` block for the same struct? Why might this be useful?

Define a struct `StringBuilder` with a field `content` of type `String`. Implement methods: 1. `new() -> StringBuilder` – creates an empty builder 2. `add(&mut self, text: &str)` – appends text to content 3. `build(&self) -> String` – returns a clone of the content

Use it in `main` to by creating a builder, adding the strings "Hello," and "World!" to it, and then building and printing the string.

Define a struct `BankAccount` with fields `owner: String` and `balance: f64`. Implement: 1. `new(owner: &str, initial: f64) -> BankAccount` 2. `deposit(&mut self, amount: f64)` – adds amount if positive 3. `withdraw(&mut self, amount: f64) -> bool` – subtracts amount if sufficient funds, returns whether successful 4. `display(&self)` – prints owner and balance

Write a `main` that creates an account owned by "Alice" with an initial balance of 100.0, deposits 50.0, withdraws 30.0, printes whether the withdrawal was successful, and displays the account information.

Fill in the blanks:

A method that needs to read but not modify the struct takes `___` as its first parameter. A method that needs to modify the struct takes `___`.

Generics

Prerequisite: Methods

Write a generic function `largest` that takes a slice `&[T]` and returns a reference to the largest element. Use appropriate trait bounds for the operation you expect to perform on the elements.

Test it from `main` with `[3, 7, 2, 9, 4]` and `["apple", "banana", "cherry"]`.

Define a generic struct `Pair<T>` with two fields `first` and `second` of type `T`. Implement a method `new` and a method `larger` that returns a reference to the larger value (require `PartialOrd`).

What is monomorphization, and why does Rust use it for generics?

Fill in the blanks:

To use `>` and `<` operators on a generic type `T`, you need the trait bound `T: ___`.

To use `println!("{}", value)` on a generic type `T`, you need the trait bound `T: ___`.

This program has a bug and does not compile. Explain why and how to fix it.

```rust
fn print_larger<T>(a: T, b: T) {
    if a > b {
        println!("{}", a);
    } else {
        println!("{}", b);
    }
}

fn main() {
    print_larger(10, 20);
}
```

Define a generic struct `KeyValue<K, V>` with fields `key: K` and `value: V`. Implement a `new` method. Create instances with `("age", 25)` and `("scores", [90, 85, 92])` in `main`. Print the key and value of each instance.

Select all statements that are true about generics in Rust:

- ☐ A. Monomorphization means the compiler generates separate code for each concrete type used.
- ☐ B. Generics have a runtime performance cost due to dynamic dispatch.
- ☐ C. You can write specialized `impl` blocks that only apply to specific concrete types.
- ☐ D. Multiple trait bounds are combined with `+`, e.g. `T: Display + PartialOrd`.

Option and Result

Prerequisite: Generics

What is `Option<T>` in Rust and what are its two variants?

Write a function `safe_divide(a: f64, b: f64) -> Option<f64>` that returns `None` if `b` is 0, otherwise returns `Some(a / b)`. In `main`, call it and use `match` to print the result or an error message.

What is the output of this program?

```rust
fn main() {
    let a: Option<i32> = Some(42);
    let b: Option<i32> = None;

    println!("{}", a.unwrap_or(0));
    println!("{}", b.unwrap_or(0));
}
```

What is `Result<T, E>` in Rust and how does it differ from `Option<T>`?

Write a function `parse_age(input: &str) -> Result<u32, String>` that parses a string to a `u32`. Return `Err` with a message if the string is not a valid number or if the number is greater than 150. Call `parse_age` from `main` with the strings "25", "200", and "abc" to print the result.

Hint: Use the `parse` method as in `input.parse::<u32>()` which returns a `Result<u32, ParseIntError>`.

Select all statements that are true about `Option` and `Result`:

- ☐ A. Calling `.unwrap()` on `None` will cause a panic at runtime.
- ☐ B. `.unwrap_or(default)` evaluates the default eagerly (even if the value is `Some`).
- ☐ C. `.unwrap_or_else(|| expr)` evaluates the closure lazily (only if the value is `None`).

☐ D. `Option<T>` and `Result<T, E>` are special compiler primitives, not regular enums.

In a main function, declare `let maybe_name: Option<&str> = Some("Alice");` and then use `if let` to print the name if it exists, or print "No name" otherwise.

What is the output of this program?

```
fn main() {
    let some_num: Option<i32> = Some(5);
    let none_num: Option<i32> = None;

    let doubled = some_num.map(|x| x * 2);
    let doubled_none = none_num.map(|x| x * 2);

    println!("{:?}", doubled);
    println!("{:?}", doubled_none);
}
```

What is the output of this program?

```
fn main() {
    let results = vec!["42", "not_a_number", "7"];
    for s in results {
        match s.parse::<i32>() {
            Ok(n) => println!("Parsed: {}", n),
            Err(e) => println!("Error: {}", e),
        }
    }
}
```

Write a function `find_first_even(numbers: &[i32]) -> Option<i32>` that returns the first even number in a slice, or `None` if there are no even numbers. Test it with `[1, 3, 4, 5, 6]` and `[1, 3, 5]`.

Write a function `average(numbers: &[f64]) -> Result<f64, String>` that returns the average of the slice. Return an `Err` if the slice is empty. Write `main` demonstrating both the success and error cases using `match`.

Closures

Prerequisite: Option and Result

What is a closure in Rust, and how does its syntax differ from a regular function?

What is the output of this program?

```rust
fn main() {
    let factor = 3;
    let multiply = |x| x * factor;
    println!("{}", multiply(5));
    println!("{}", multiply(10));
}
```

This program has a bug and does not compile. Explain why and how to fix it.

```rust
fn main() {
    let identity = |x| x;
    let s = identity("hello");
    let n = identity(42);
}
```

Write a closure that captures a mutable variable `count` and increments it each time it is called. Call it three times and print the final count.

What is the difference between `.unwrap_or(default)` and `.unwrap_or_else(|| expr)` in terms of evaluation?

Select all statements that are true about closures vs regular functions:

- ☐ A. Closures can capture variables from their enclosing scope.
- ☐ B. Regular functions can capture variables from their enclosing scope.
- ☐ C. Closures always require type annotations on their parameters.
- ☐ D. Closure parameter/return types are inferred from the first use and then fixed.

Write a function `apply_twice` that takes an `i32` value and a closure that maps `i32 -> i32`, and returns the result of applying the closure twice. Test it by calling it from `main` with a closure that doubles a number (e.g., 3 becomes 6), and then calling `apply_twice` with that closure and the number 3 (so 3 becomes 12).

Hint: Use the `Fn` trait bound to specify the closure type as in `Fn(i32) -> i32`.

Create a vector of strings `["banana", "apple", "cherry"]` and sort it by string length (shortest first) using a closure with `.sort_by`. Print the result.

What does the `move` keyword do when placed before a closure? When is it needed?

Iterators

Prerequisite: Closures

What method does the `Iterator` trait require you to implement, and what does it return?

What are the outputs of Programs 1 and 2?

Hint: They are not the same.

```
// Program 1
fn main() {
    let v = vec![1, 2, 3, 4, 5];
    let _mapped = v.iter().map(|x| {
        println!("processing {}", x);
        x * 2
    });
    println!("done");
}
```

```
// Program 2
fn main() {
    let v = vec![1, 2, 3, 4, 5];
    let double_v: Vec<_> = v.iter().map(|x| {
        println!("processing {}", x);
        x * 2
    }).collect();
    println!("done");
}
```

Using iterator methods, take a `Vec<i32>` of `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`, filter to keep only even numbers, square each one, and collect into a `Vec<i32>`. Print the result.

Select all statements that are true about ranges as iterators:

- ☐ A. `1..5` produces values 1, 2, 3, 4.
- ☐ B. `1..=5` produces values 1, 2, 3, 4, 5.
- ☐ C. `1.0..5.0` is a valid iterator over floating-point numbers.
- ☐ D. `'a'..'z'` is a valid range over characters.

Implement a custom iterator `Countdown` that counts down from a given starting number to 1.

Define the struct and implement the `Iterator` trait for it. In `main`, create a `Countdown` starting from 5 and use a `for` loop to print each value.

Select all statements that are true about iterator adapters and consumers:

- ☐ A. `.map()` and `.filter()` are adapters that return new iterators.
- ☐ B. `.collect()` and `.fold()` are consumers that produce a final value.
- ☐ C. Adapters execute eagerly as soon as they are called.
- ☐ D. You must consume an iterator (e.g., with `.collect()`) for adapters to execute.

What is the output of this program?

```rust
fn main() {
    let v = vec![10, 20, 30];
    let mut iter = v.iter();
    println!("{:?}", iter.next());
    println!("{:?}", iter.next());
    println!("{:?}", iter.next());
    println!("{:?}", iter.next());
}
```

What is the output of this program?

```rust
fn main() {
    let first_five: Vec<i32> = (1..).filter(|x| x % 3 == 0).take(5).collect();
    println!("{:?}", first_five);
}
```

Use `.enumerate()` to print each element of `["a", "b", "c"]` with its index, like:

```
0: a
1: b
2: c
```

What is the difference between `.iter()`, `.iter_mut()`, and `.into_iter()` on a `Vec<T>`?

Use a range and `.collect()` to create a `Vec<u32>` containing `[1, 2, 3, 4, 5]`. Then use `.collect()` again to create a `String` from the characters `['H', 'i', '!']`.

Iterators and Closures Combined

Prerequisite: Iterators

In a main function, use `.fold()` to compute the sum of squares of numbers from 1 to 5. Print the result.

What is the difference between `.fold()` and `.reduce()` on an iterator?

What is the output of this program?

```
fn main() {
    let has_negative = [3, -1, 4, 1, 5].iter().any(|x| *x < 0);
    let all_positive = [3, 1, 4, 1, 5].iter().any(|x| *x < 0);
    println!("{}", has_negative);
    println!("{}", all_positive);
}
```

Given two vectors `v1 = vec![1.0, 2.0, 3.0]` and `v2 = vec![4.0, 5.0, 6.0]`, use `.zip()`, `.map()`, and `.sum()` to compute their dot product (sum of element-wise products). Print the result.

Write a function `is_prime(n: u32) -> bool` using the `.any()` iterator method. A number is prime if it is greater than 1 and no number from 2 to its square root divides it evenly.

Test it in `main` by printing whether 7, 10, and 13 are prime.

What is the output of this program?

```rust
fn main() {
    let result: Vec<String> = (1..=5)
        .filter(|x| x % 2 == 1)
        .map(|x| format!("#{}", x))
        .collect();
    println!("{:?}", result);
}
```

What is the output of this program?

```rust
fn main() {
    let max = vec![3, 7, 2, 9, 4]
        .into_iter()
        .reduce(|a, b| if a > b { a } else { b });
    println!("{:?}", max);

    let empty: Vec<i32> = vec![];
    let result = empty.into_iter().reduce(|a, b| a + b);
    println!("{:?}", result);
}
```

Use `.for_each()` to print each number from 1 to 5, each on its own line.

Write a program that takes a string `"the quick brown fox jumps over the lazy dog"` and uses iterator methods to count the number of words that have more than 3 characters. Print the count.

What is the output of this program?

```rust
fn main() {
    let sum: i32 = (1..=10)
        .filter(|x| x % 3 == 0)
        .map(|x| x * x)
        .sum();
    println!("{}", sum);
}
```

Use `.partition()` to split the numbers 1 through 10 into two vectors: one with even numbers, one with odd numbers. Print both.

Modules and Crates

Prerequisite: None

In Rust, are items inside a module public or private by default? How do you make an item public?

This program has a bug and does not compile. Explain why and how to fix it.

```
mod math {
    fn add(a: i32, b: i32) -> i32 {
        a + b
    }
}

fn main() {
    println!("{}", math::add(3, 4));
}
```

Create a nested module structure: `shapes::circle` with a public function `area(radius: f64) -> f64`. Call it from `main` using the full path and also with a `use` statement so you can call `circle::area(5.0)` instead of the full path.

Hint: Use the `std::f64::consts::PI` constant.

This program has a bug and does not compile. Explain why and how to fix it.

```
mod account {
    pub struct BankAccount {
        pub owner: String,
        balance: f64,
    }
}

fn main() {
    let acct = account::BankAccount {
        owner: String::from("Alice"),
        balance: 100.0,
    };
}
```

Select all statements that are true about crates in Rust:

☐ A. A binary crate compiles to an executable and has a `main()` function.
☐ B. A library crate compiles to reusable code and does not have a `main()` function.
☐ C. External crates are added as dependencies in `Cargo.toml`.
☐ D. A package can contain at most one binary crate.

Fill in the blanks:

To refer to the parent module from within a nested module, use the keyword `___`.

To refer to the crate root in an absolute path, use the keyword `___`.

Given a module with several public functions, use a single `use` statement with curly braces to import multiple items. Write a module `math` with `pub fn add`, `pub fn subtract`, and `pub fn multiply`, then import all three with one `use` statement and call each function from `main`.

Predict the output of this program.

```
fn greet() -> &'static str {
    "hello from root"
}

mod inner {
    pub fn call_parent() -> &'static str {
        super::greet()
    }
    fn greet() -> &'static str {
        "hello from inner"
    }
}

fn main() {
    println!("{}", inner::call_parent());
}
```

Create a module `geometry` with a struct `Circle` where its field `radius` is private. Provide:
1. A public constructor `new(radius: f64) -> Circle` 2. A public method `area(&self) -> f64`

Create a `main` function that creates a `Circle` with radius 5.0, prints the area.

How do you add an external crate (e.g., `rand`) as a dependency to your Rust project? Name two ways.

Fill in the blanks:

In semantic versioning (major.minor.patch), a `___` version change indicates new backwards-compatible features. A `___` version change indicates backwards-compatible bug fixes. A `___` version change indicates backwards-incompatible changes.

Predict the output of this program.

```
fn secret() -> &'static str {
    "found me in root"
}

mod outer {
    mod inner {
        pub fn secret() -> &'static str {
            "found me in inner"
        }
    }

    pub fn reveal() -> &'static str {
        inner::secret()
    }

    fn secret() -> &'static str {
        "found me in outer"
    }
}

fn main() {
    println!("{}", outer::reveal());
}
```

Tests

Prerequisite: Modules and Crates

What attribute do you place above a function to mark it as a test function in Rust? How do you run tests?

Write a function `is_even(n: i32) -> bool` and a test module with tests for even and odd numbers using `assert!` and `assert_eq!`.

Select all statements that are true about test assertion macros:

- ☐ A. `assert!(expr)` passes if `expr` evaluates to `true`.
- ☐ B. `assert_eq!(a, b)` passes if `a == b`.
- ☐ C. `assert_ne!(a, b)` passes if `a == b`.
- ☐ D. All assertion macros can take an optional custom failure message.

Write a function `clamp(value: i32, min: i32, max: i32) -> i32` that returns: - `min` if `value < min` - `max` if `value > max` - `value` otherwise

Write a comprehensive test module covering normal case, when value is below min or above max and when min equals max.

```
fn clamp(value: i32, min: i32, max: i32) -> i32 {
    // implement this

}

#[cfg(test)]
mod tests {
    use super::*;

    // test when value is within range

    // test when value is below min

    // test when value is above max

    // test when min equals max

}
```

What are documentation tests in Rust? How does `cargo test` handle them?

Select all statements that are true about organizing tests in Rust:

- ☐ A. Unit tests typically live in the same file as the code they test.
- ☐ B. The `#[cfg(test)]` attribute ensures test code is only compiled during `cargo test`.
- ☐ C. `use super::*;` inside a test module imports all items from the parent module.
- ☐ D. Tests must always be in a separate `tests/` directory.

Write a function `remove_negatives(v: &[i32]) -> Vec<i32>` that returns a new vector with only the non-negative numbers. Write at least three tests: empty input, all negative input, and mixed input.